

TD1

Des octets au code ASCII

1. On considère l'information suivante, codée en binaire.

a) Traduire cette information en hexadécimal. Veuillez d'abord à découper cette information en unités interprétables.

Pour pouvoir traduire cette information en hexadécimal, il faut d'abord découper cette chaîne en sous-chaînes de 8 bits, c'est-à-dire en octets, puisque l'information est interprétée de cette manière par l'ordinateur. On traduit ensuite chaque octet (suite de 8 positions binaires) en 2 chiffres hexadécimaux.

On peut également découper plus finement en sous-chaînes 4 bits. Il devient alors possible de chercher la correspondance entre ces sous-chaînes de 4 bits et le chiffre hexadécimal qu'elles transcrivent.

Présentation :

- directement sous la chaîne binaire, ou bien sous forme de tableau découpé en octets ou en sous-chaînes de 4 bits.
- toujours écrire l'hexadécimal en 2 chiffres lorsqu'il s'agit d'un octet.

b) Observer attentivement cette suite binaire. Si celle-ci est logée en mémoire, que pouvez-vous dire des *octets* qui la composent ?

En découpant la chaîne en octets (chaînes de 8 bits), on peut constater que le bit de poids fort, c'est-à-dire le bit le plus à gauche, est toujours à 0.

c) Cette information peut-elle être décodée en tant que texte ASCII ? Justifier.

Puisque le bit de poids fort est toujours à 0, cela signifie que le 8^e bit de l'octet n'est pas utilisé. L'encodage de ce texte est donc sur 7 bits, ce qui est également le cas de la table ASCII. Ce texte peut donc bien être décodé en ASCII.

Nous pouvons aussi ajouter que la représentation hexadécimale de chacun des octets de ce texte commence par un chiffre entre 0 et 7, ce qui est également une des caractéristiques des tables 7 bits telles que ASCII.

- d) Décoder l'ensemble (ou ce que vous pouvez) de ce texte à l'aide de la table ASCII vue en cours.

ASCII[space]definit[space]128[space]codes[space]a[space]7[space]bits.
[LF]

[space] correspondant au caractère espace, et [LF] au retour à la ligne.

2. Soit la représentation suivante des octets d'un fichier présentés en hexadécimal.

- a) Ce fichier ne semble pas être un fichier ASCII. Expliquer pourquoi à partir de vos observations.

Contrairement au texte de la question (1), ce texte contient des octets dont la représentation hexadécimale commence par un chiffre supérieur à 7 : E2, 80, 99, A9, A8.

- b) Si certains octets sont tout de même décodables, les décoder.

L_____t__[space]suit[space]le[space]printemps[space]et[space]pre__c__de[space]
e]l_____automne.[LF]

3. Considérons le texte suivant :

Charlemagne devint le roi des Francs en 768 et empereur d'Occident en 800.

S'il est enregistré dans un fichier au format texte brut, le fichier sera-t-il un fichier ASCII ? Justifier.

À première vue, ce texte peut bien être enregistré en ASCII puisqu'il contient des caractères qui semblent être contenus dans la table ASCII, à savoir des majuscules et des minuscules non diacritées, des chiffres, ainsi qu'une apostrophe.

4. Peut-on coder le texte suivant en ASCII ?

Son père était Pépin le Bref.

Contrairement au texte de la question 3, ce texte contient des caractères diacrités : « è » dans « père » et « é » dans « était » et « Pépin ». La table ASCII ne contient aucun diacritique ni caractère diacrité. Il est donc impossible de coder ce texte en ASCII.

5. On a constitué un répertoire de 327 symboles. On souhaite établir un jeu de caractères codés à partir de ce répertoire de symboles de sorte que le code distingue chacun des éléments.

De combien de positions binaire devra-t-on disposer pour coder ce répertoire de symboles ? Et combien d'octets ? Décrire précisément son raisonnement.

Pour distinguer chaque symbole, il faudrait que notre code comporte 327 éléments et donc 327 combinaisons binaires distinctes.

Nous avons vu qu'une table 7 bits telle que ASCII permet d'encoder 128 caractères (2^7). En utilisant le 8^e bit, nous pouvons encoder le double de caractères, soit 256 caractères (2^8).

Toutefois, si nous avons un répertoire de 327 symboles, nous ne pourrions pas l'encoder en 8 bits, puisque $327 > 256$. Si nous ajoutons encore un bit, nous doublons encore les possibilités de combinaisons binaires distinctes : $2^9 = 512$. Cette fois, il serait possible d'encoder nos 327 symboles, puisque $327 < 512$. Il faut donc 9 positions binaires pour coder notre répertoire de 327 symboles.

Un octet contient 8 positions binaires. Il faudra donc 2 octets pour nos 9 positions binaires (et il restera 7 positions inutilisées sur ce deuxième octet).

6. Recopier le texte que vous avez décodé à la question 1.d) dans un *éditeur de texte*, et sauvegardez-le en *texte brut*. Ouvrez ensuite ce fichier à l'aide de votre éditeur hexadécimal. Vérifiez que la représentation hexadécimale correspond bien à celle que vous avez trouvée en 1.a), sinon, y remédier.
7. Recopier le texte de la question 3 dans un fichier, puis à l'aide de l'outil adéquat :
 - a) Donner son image en hexadécimal.
 - b) Donner son image en binaire.

Ici, il suffit de prendre une capture d'écran (ou *screenshot*) de l'image en hexadécimal puis binaire du texte. Pour cela, il faut d'abord bien recopier le texte, sans se tromper ! Ensuite, l'ouvrir dans un éditeur hexadécimal, ou bien, si l'éditeur dont nous disposons ne permet pas d'afficher le code en binaire (c'est le cas du plug-in HEX Editor de Notepad++), utiliser un site tel que <https://www.convzone.com/hex-to-binary/>.

8. Observer attentivement les fichiers intitulés `autoportrait_bacchus` du dossier `exemples_fichiers`.
 - a) Inspecter en particulier les miniatures/icônes des fichiers et leurs extensions. Que pouvez-vous en dire ?

Les icônes des fichiers sont différentes pour chacun des fichiers (voir capture). Ces différences semblent correspondre au format, et donc à l'extension des fichiers, puisqu'on voit par exemple une croche et les lettres « wav » dans l'icône du fichier `.wav` qui est normalement un fichier son. De même, je vois des lignes pour le fichier `.txt` et un dessin pour le fichier `.png`, qui est normalement un format image.

Ces différences semblent aussi correspondre au programme qui permet habituellement à ouvrir les fichiers, puisque l'on voit le logo de Word pour le fichier `.docx`, le logo Writer pour le fichier `.odt` et le logo de Adobe pour le `.pdf`.

- b) Arrivez-vous à tous les ouvrir ? Que se passe-t-il et pourquoi ?

Je peux ouvrir sans problème les fichiers `.txt`, `.epub` (qui s'ouvre comme un zip), `.docx`, `.odf` et `.pdf`. En revanche, lorsque j'essaie d'ouvrir le fichier `.png`, je vois l'erreur suivante : « **Fatal error reading PNG image file: Not a PNG file** ». Je n'arrive pas non plus à ouvrir le fichier `.wav`, qui provoque l'erreur suivante : « **An error occurred. An audio or video stream is not handled due to missing codecs. You might need to install additional**

plugins to be able to play some types of movies ». Dans le premier cas, la machine semble reconnaître que ce fichier n'est pas une image. Dans le second cas, il manque quelque chose à la machine pour l'interpréter correctement. Si ces fichiers sont codés comme des textes, il est normal qu'un logiciel qui permet d'ouvrir des images et un autre des sons et des vidéos ne puissent pas les ouvrir.

9. Observer cette fois les fichiers intitulés `caravage_1.txt` et `caravage_2.txt`.

- a) Ouvrir ces deux fichiers dans un éditeur de texte classique. Lequel est en ASCII ? Comment le savez-vous ?

Le fichier `caravage_1.txt` contient des caractères qui ne se trouvent pas dans la table ASCII : des caractères diacrités (« é », « à », « è ») ainsi que le « œ ». Ce n'est pas le cas du fichier `caravage_2.txt`, dans lequel tous ces caractères ont été remplacés par des équivalents non diacrités ou par « oe » dans le cas de « œ ». C'est donc le second fichier qui est en ASCII.

- b) Repérer les caractères de contrôle dans la table ASCII vue en cours. Que pouvez-vous dire sur leur point de code ?

Dans la table ASCII vue en cours, les caractères de contrôle se trouvent exclusivement dans les deux premières colonnes. Leurs points de code se situent donc entre 00 et 1F. Autrement dit, leurs points de code hexadécimaux commencent forcément par un 0 ou par un 1. S'ils commencent par un 2 ou un 7, il s'agit de caractères imprimables.

- c) Trouver tous les caractères de contrôle du fichier `caravage_2.txt`.

Les caractères de contrôle de ce fichier sont :

- 0A : LF (*Line Feed*), le retour à la ligne
- 09 : HT (*Horizontal Tabulation*), la tabulation horizontale